Comparison of related research papers with LeakLens: checked version

Software Engineering Minor Project. Replaces the earlier "Comparative Literature Review Matrix". Other papers: what we could read ourselves on 13 November 2026. LeakLens: only what a logged run or a test shows.

MEASURED = a number from a logged run in the repository. **BUILT** = the feature exists and is tested; it is a difference in what the tools do, not proof of better accuracy. **NOT SHOWN** = the earlier table claimed it, and it is not true or not demonstrated yet. **not checked** = we could not open the paper to confirm this detail.

The one measured comparison is with Yang et al. (ASE 2022), which is the detection engine inside LeakageDetector 2.0. None of the other three tools could be run: one needs a paid LLM service, one has no public code that we found, one needs a GPU.

1. Research paper	2. What the paper does	3. Limits (as far as we could check)	4. How LeakLens compares, and what backs it
LeakageDetector 2.0 Truong, Zhang, Marchareddy, Lee, Busold, Socas, AlOmar. arXiv:2509.15971, 2025. *Checked: arXiv full text read.*	• VS Code extension for Jupyter notebooks; detects "mainly Overlap, Preprocessing and Multi-test leakage". • Detection is Yang et al.'s static analyser, shipped as native binaries; Docker is optional. • Quick fixes, with optional GitHub Copilot suggestions.	• No group or temporal leakage mentioned. • Static only: cannot see what is in the data. • Its multi-test fix inserts a placeholder call, load_test_data(). • Reports no accuracy of its own: "we did not assess the detection accuracy of the tool separately"; it relies on Yang et al.'s numbers. • No measurement of how much a leak changes the score.	• **MEASURED** Its detection engine (Yang et al.) and LeakLens were run on the same 79 held-out files, scored per file: preprocessing 10 vs 21 of 22, overlap 7 vs 7 of 9, multi-test 3 vs 5 of 6. No false alarm from either. The files are our own benchmark. • **BUILT** Looks for group and temporal leakage, which this tool does not. • **BUILT** Also runs the code and checks the data (repeated entities, duplicate rows). • **BUILT** Re-runs the script with a fix and reports the score change, for about half of the leaks. • **NOT SHOWN** "Lighter because no Docker": Docker is optional for this tool too. • **NOT SHOWN** "0.95 to 0.43 AUPRC": that figure is from CalibraEEG, where LeakLens found nothing.
Static Slicing and Structured LLM Prompts Draoui, Mkaouer, Newman. Proc. ACM Softw. Eng. (FSE), 2026. *Checked: abstract only (Sept. 2026); the publisher blocks the full text.*	• Cuts a notebook into train/evaluate slices and asks an LLM about each (abstract). • Abstract: F1 better than prior work by 22% (preprocessing) and 15% (overlap). • **not checked** 451 notebooks, GPT-4o-mini, 5-step checklist: stated in the original table, not confirmed by us.	• Covers preprocessing and overlap (abstract). • Needs an LLM service and a running slicer service to use. • **not checked** "14.3 seconds per notebook" and "cannot detect duplicate records": not confirmed by us.	• **NOT SHOWN** No head-to-head result: the tool needs a paid LLM key and its package is access-restricted, so it was not run. • **BUILT** No LLM, no per-use cost, same answer every run. • **MEASURED** Static analysis took a median of 2.6 ms per file on our files (a laptop). Their time is on their notebooks, so this is not a like-for-like speed comparison. • **BUILT** Finds duplicate rows by checking the data at run time. • **NOT SHOWN** "Catches sliding-window leaks": not shown; the one real case tried (CalibraEEG) was missed.
Transfer Learning, Active Learning, or Low-Shot Prompting? Alturayeif, Hassine. PeerJ Computer Science 11:e2730, 2025. *Checked: could not be opened (publisher blocks automated access). Every detail in this row is **not checked**.*	• **not checked** Trains models to label a code snippet as leaky or not (as described in the original table). • **not checked** 1,904 snippets; F2 = 0.72 with active learning.	• **not checked** Works on separate snippets, not whole notebooks. • **not checked** Says "leaky or not" without pointing to a line. • A different kind of work from ours: a trained classifier, not an analysis of how data flows.	• **NOT SHOWN** No head-to-head result: no public code was found to run. • **BUILT** Analyses whole scripts and notebooks and reports the line. • **MEASURED** On our held-out benchmark: no false alarm, 65 of 73 leaks found (first look). • **NOT SHOWN** "90% precision or better": true only on our own generated files. No real notebook has been labelled, so there is no precision figure for real code.

1. Research paper	2. What the paper does	3. Limits (as far as we could check)	4. How LeakLens compares, and what backs it
scicode-lint Samsonau. arXiv:2603.17893, 2026. *Checked: arXiv full text read.*	• 66 patterns for scientific Python; a small local LLM checks each pattern. • On Yang et al.'s human-labelled Kaggle notebooks: preprocessing leakage 65% precision at 100% recall; multi-test 80% precision at 18% recall. • Has an overlap pattern for sliding windows (stride smaller than the window).	• "Overlap leakage remains undetected" on those notebooks (the paper gives no 0.0% figure). • 24% precision for all "critical" findings on research code, judged by an LLM (not only leakage). • 11% precision for its AI-training patterns on held-out papers, judged by an LLM. • Needs "a workstation GPU with 20GB+ VRAM". • No measurement of how much a leak changes the score.	• **NOT SHOWN** No head-to-head result: it needs a GPU, which this project's rules reserve for another project. Its numbers above are on real, human-labelled notebooks; ours are on generated files. • **BUILT** Runs on a laptop CPU with no LLM. • **MEASURED** Overlap on our held-out files: 7 of 9 from code, 9 of 9 with the run. Not the same data as theirs. • **BUILT** Group leakage, which it has no pattern for. • **NOT SHOWN** Sliding-window overlap: scicode-lint has a pattern for it; LeakLens does not yet.

Check of each claim the earlier table made about LeakLens

"True" means it can be shown from the repository today. Evidence: results/ and tests/ in github.com/bytes06runner/LeakLens.

Claim in the earlier table	Status	What the evidence says
Detects six leak types, including group and temporal	**True**	Rules and tests for all six; TEST first look finds leaks of every type. scicode-lint also covers part of temporal, so do not say "only we do".
Hybrid: static analysis plus an instrumented run that hashes rows and tracks entity ids	**True**	Built and tested; on TEST the run added the 2 duplicate-row leaks. It named the patient id correctly only in the second look, after a fix.
Reports the score drop after an automatic fix	**Partly**	Works for six kinds of fix: 34 of 70 located TEST leaks. None for multi-test, resampling, pandas statistics, forward-looking features.
Example "AUPRC 0.95 to 0.43"	**Not true for LeakLens**	That is the CalibraEEG measurement. LeakLens found nothing on the CalibraEEG code. Remove it.
Command-line tool	**True**	leaklens scan, with JSON and SARIF output.
Pre-commit hook	**True since 13 November**	Built today; a real pre-commit run failed a leaky file and passed a clean one.
GitHub Action / CI gating	**True since 13 November**	Built today; ran on GitHub and uploaded 8 findings to code scanning.
No Docker needed, unlike LeakageDetector 2.0	**Not an advantage**	LeakageDetector 2.0 no longer needs Docker either (its paper).
Catches sliding-window leaks	**Not shown**	Missed on CalibraEEG; scicode-lint has a pattern for this and LeakLens does not.
Deterministic, no LLM, no GPU, no per-use cost	**True**	By design; tests check the output is identical on repeat runs.
Analyses whole notebooks and points at the line	**True**	The frozen scoring rule requires the right line; TEST results are under that rule.
At least 90% precision	**Only on our own files**	No false positive on DEV or TEST of the injected benchmark. No figure exists for real notebooks: none is labelled.
Reliable overlap detection	**On our own files**	7 of 9 from code, 9 of 9 with the run. Synthetic copies made by SMOTE cannot be confirmed at run time.
Better than these four papers	**Not shown**	None of the four was run against LeakLens. One measured comparison exists: Yang et al., the engine inside LeakageDetector 2.0, on our own files.

What can be said to the supervisor. LeakLens does things the four papers do not describe: it looks for group leakage, it checks the data by running the code, and it measures the score change for a located leak. On our own held-out benchmark it found at least as many leaks of every shared kind as the Yang et al. engine, with no false alarm. It has not been compared head-to-head with these four tools, and it has no accuracy figure on real notebooks yet.